

Chương 7.2
Entity Framework - LINQ
(SQL Server)

Biên soạn: ThS. TRẦN KIM HƯƠNG

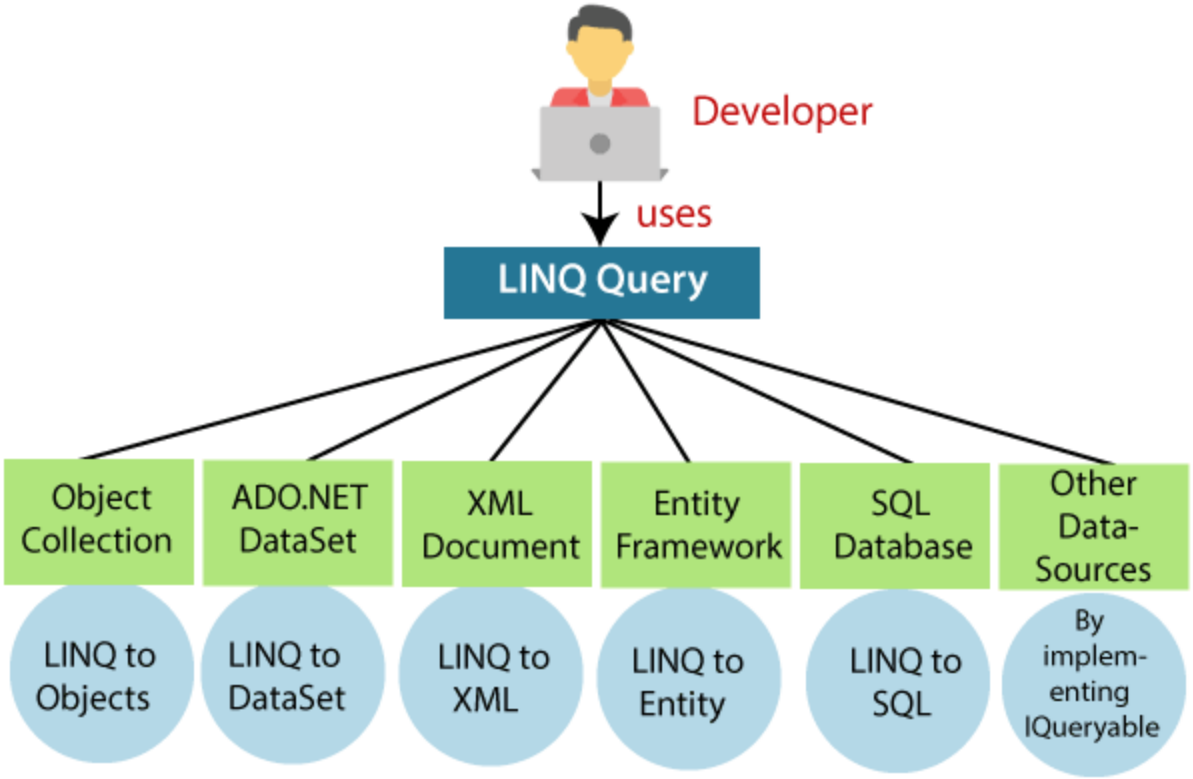
tkhuong@dthu.edu.vn

Outline

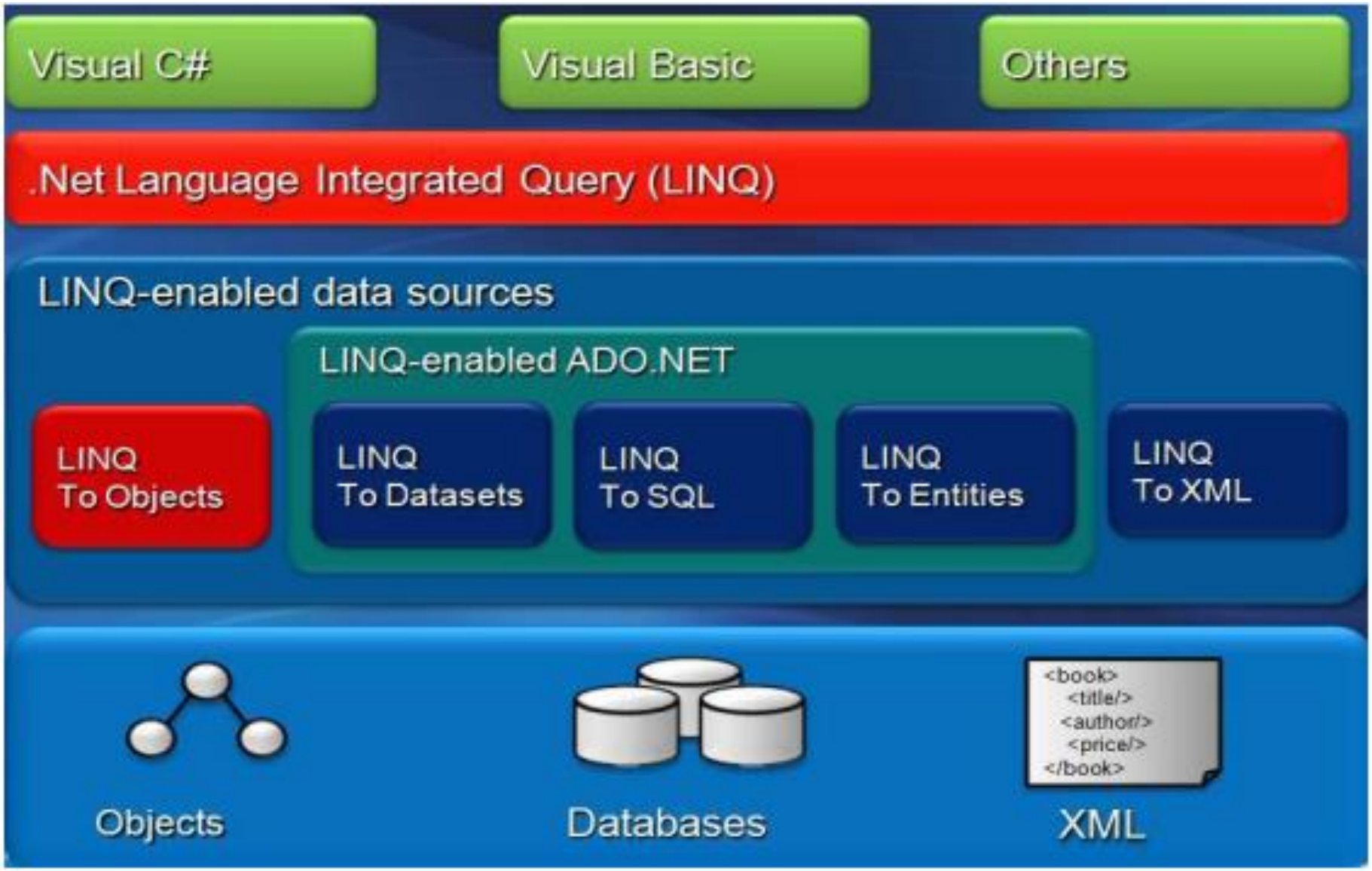
- LINQ là gì
- Cú pháp truy vấn LINQ
- Các phương thức LINQ to Entity
- Hướng dẫn thực hành LINQ to Entity
- Bài tập thực hành LINQ to Entity
- Bài tập thực hành Xây dựng website BookShop Online

What is LINQ?

- LinQ (Language Integrated Query) là cú pháp truy vấn có cấu trúc được xây dựng trong C# để truy xuất dữ liệu từ các nguồn dữ liệu khác nhau như collections, ADO.NET DataSet, XML Docs, web service, MS SQL Server và các database khác.



Kiến trúc LINQ



Example LINQ to Object

- EX1. Using foreach in List

```
// Find student with age >=23 --> 30
List<Student> result = new List<Student>();
foreach (var st in lstInfoStudent)
{
    if(st.StudentAge >=23 && st.StudentAge <= 30)
    {
        result.Add(st);
    }
}
return View(result);
```

```
<h2>LinQ Student</h2>
@foreach (var item in Model)
{
    <p>
        Id: @item.StudentId <br />
        Name: @item.StudentName<br />
        Age: @item.StudentAge
    </p>
}
```

```
public class Student
{
    4 references
    public string StudentId { get; set; }
    4 references
    public string StudentName { get; set; }
    6 references
    public int StudentAge { get; set; }
}
```

```
List<Student> lstInfoStudent = new List<Student>()
{
    new Student
    {
        StudentId = "S001",
        StudentName = "Kim Huong",
        StudentAge = 30
    },
    new Student
    {
        StudentId="S002",
        StudentName="Huong Tran",
        StudentAge= 20
    },
    new Student
    {
        StudentId="S003",
        StudentName="Hung Thanh",
        StudentAge= 25
    },
    new Student
    {
        StudentId="S004",
        StudentName="Tran Thanh",
        StudentAge= 18
    },
};
```

Example LINQ to Object

- EX2. Using LINQ

```
// Using LINQ
List<Student> result = new List<Student>();
result = lstInfoStudent.Where(t => t.StudentAge >= 23 && t.StudentAge <= 30).ToList();
return View(result);
```

```
Student result = new Student();
result = lstInfoStudent.Where(t => t.StudentAge >= 23 && t.StudentAge <= 30).FirstOrDefault();
return View(result);
```

Why LINQ?

- Advantages of LINQ
  - Ngôn ngữ quen thuộc: không phải học ngôn ngữ truy vấn mới cho từng nguồn dữ liệu khác nhau.
  - Ít code: giảm số lượng code được viết so với truy vấn truyền thống
  - Code dễ đọc: code đơn giản, rõ ràng, dễ hiểu
  - Chuẩn hóa để truy vấn nhiều nguồn dữ liệu: dùng 1 cú pháp LINQ để truy vấn trên nhiều nguồn dữ liệu khác nhau.
  - Biên dịch an toàn: nó kiểm tra kiểu của đối tượng tại thời điểm biên dịch.

LINQ to Entity

- Có 2 cách viết câu truy vấn LINQ

```
private NewsEntities _dbcontext = new NewsEntities();

* LINQ Method Synax
public ActionResult GetArticleByAuthor(string authorId)
{
    var result = _dbcontext.BaiViets.Where(x => x.MaTG ==
authorId).ToList();
    return View(result);
}

* LINQ Query Synax
public ActionResult GetArticleByAuthor1(string authorId)
        {
            var result = from news in _dbcontext.BaiViets
                         where news.MaTG == authorId
                         select news;
            return View(result);
        }
```

LINQ Query Syntax

- LINQ có 2 cú pháp truy vấn là Query syntax và Method syntax, sử dụng thư viện System.Linq
- LINQ Query Syntax

```
var lists = from <Biến lưu thông tin từng phần tử> in <Nguồn dữ liệu>
[<Phép toán truy vấn: where, join ... in, order by...> Biểu thức lambda]
select <Biến lưu thông tin từng phần tử>
```

- Ví dụ

```
private DBQLNVDataContext _dbContext = new DBQLNVDataContext();
var result = from nv in _dbContext.NhanViens
                        select nv;
            dgvNV.DataSource = result;
```

https://www.tutorialspoint.com/linq/linq_query_operators.htm

LINQ Query Syntax

- **Ví dụ**

```
private DBQLNVDataContext _dbContext = new DBQLNVDataContext();

var result = from nv in _dbContext.NhanViens
             select nv;


var result = from nv in _dbContext.NhanViens
              where nv.MaCV == "P"
              select new
               {
                      MaNhanVien = nv.MaNV,
                      hotenNV = nv.HoNV,
                      tenNV = nv.TenNV,
                      luong = nv.Luong,
                      maCV = nv.MaCV,
              };
```

LINQ Query Syntax

- **Ví dụ. Linq join table**

```
private DBQLNVDataContext _dbContext = new DBQLNVDataContext();
```

```
var result = from nv in _dbContext.NhanViens
             join cv in _dbContext.ChucVus on nv.MaCV equals cv.MaCV
             join bp in _dbContext.Bophans on nv.MaBP equals bp.MaBP
             select new
             {
                  hoNV = nv.HoNV,
                  tenNV = nv.TenNV,
                  luong = nv.Luong,
                  chucVu = cv.TenCV,
                  boPhan = bp.TenBP,
              };
```

LINQ Query Syntax

- **Ví dụ. Linq Left join table**

```
private DBQLNVDataContext _dbContext = new DBQLNVDataContext();

var result = from nv in _dbContext.NhanViens
             join cv in _dbContext.ChucVus on nv.MaCV equals cv.MaCV
                  into nv_cv
             join bp in _dbContext.Bophans on nv.MaBP equals bp.MaBP
                  into nv_bp
             from nc in nv_cv.DefaultIfEmpty()
             from nb in nv_bp.DefaultIfEmpty()
             select new
             {
              tenNV = nv.TenNV,
               chucVu = nc != null ? nc.TenCV : "",
               boPhan = nb != null ? nb.TenBP : "",
                Fullname = nv.HoNV + " " + nv.TenNV
             };
```

LINQ Extension Methods (1)

LINQ Extension Methods
First()
FirstOrDefault()
Single()
SingleOrDefault()
ToList()
Count()
Min()
Max()
Last()
LastOrDefault()
Average()

https://www.entityframeworktutorial.net/querying-entity-graph-in-entity-framework.aspx

LINQ Extension Methods (2)

- The following are some of the standard query operators (or extension methods) that can be used with LINQ-to-Entities queries.

LINQ Extension Methods		
.Where(e=>đk)	.GroupBy(e=>bthuc)	.OrderBy(e=>bthuc)/ .OderByDescending
.Take(sluong)/ .TakeWhile(e=>đk)	.Skip(sluong)/ .SkipWhile(e=>đk)	.Single(e=>đk)/ .SingleOrDefault()
.First()/ FirstOrDefault()	.Last()/ LastOrDefault()	.Find()
.ToList()	.Count(e=>bthuc)	.Min()
.Max()	.Average()	.Sum()

LINQ Method Syntax

- **VD.**

LINQ Query Syntax:

```
using (var ctx = new SchoolDBEntities())
{
    var student = (from s in ctx.Students
                   where s.StudentName == "Bill"
                   select s).FirstOrDefault<Student>();
}
```

LINQ Method Syntax:

```
using (var ctx = new SchoolDBEntities())
{
    var student = ctx.Students
                     .Where(s => s.StudentName == "Bill")
                     .FirstOrDefault<Student>();
}
```

```
SELECT TOP (1)
[s].[StudentId], [s].[DoB], [s].[FirstName], [s].[GradeId],
[s].[LastName], [s].[MiddleName]
FROM [Students] AS [s]
WHERE [s].[FirstName] = N'Bill'
```

LINQ Method Syntax

VD. GroupBy

LINQ Query Syntax:

```
using (var ctx = new SchoolDBEntities())
{
    var students = from s in ctx.Students
                   group s by s.StandardId into studentsByStandard
                   select studentsByStandard;

    foreach (var groupItem in students)
    {
        Console.WriteLine(groupItem.Key);

        foreach (var stud in groupItem)
        {
            Console.WriteLine(stud.StudentId);
```

LINQ Method Syntax:

```
using (var ctx = new SchoolDBEntities())
{
    var students = ctx.Students.GroupBy(s => s.StandardId);

    foreach (var groupItem in students)
    {
        Console.WriteLine(groupItem.Key);

        foreach (var stud in groupItem)
        {
            Console.WriteLine(stud.StudentId);
        }

    }
}
```

LINQ Method Syntax

VD. GroupBy

- *The above query will execute the following database query:*

```
SELECT [Project2].[C1] AS [C1],
           [Project2].[StandardId] AS [StandardId],
           [Project2].[C2] AS [C2],
           [Project2].[StudentID] AS [StudentID],
           [Project2].[StudentName] AS [StudentName],
           [Project2].[StandardId1] AS [StandardId1]
FROM ( SELECT [Distinct1].[StandardId] AS [StandardId],
           1 AS [C1], [Extent2].[StudentID] AS [StudentID],
           [Extent2].[StudentName] AS [StudentName],
           [Extent2].[StandardId] AS [StandardId1],
           CASE WHEN ([Extent2].[StudentID] IS NULL) THEN CAST(NULL AS int) ELSE 1 END AS [C2]
           FROM (SELECT DISTINCT [Extent1].[StandardId] AS [StandardId]
                      FROM [dbo].[Student] AS [Extent1] ) AS [Distinct1]
                      LEFT OUTER JOIN [dbo].[Student] AS [Extent2] ON ([Distinct1].[StandardId] =
[Extent2].[StandardId]) OR (([Distinct1].[StandardId] IS NULL) AND ([Extent2].[StandardId] IS NULL))
) AS [Project2]
ORDER BY [Project2].[StandardId] ASC, [Project2].[C2] ASC
```

LINQ To Entity Framework

Chú ý

Có 2 kịch bản để thực thi truy vấn:

- **Trì hoãn thực thi** (Deferred Execution): việc thực thi truy vấn chỉ khi thực hiện foreach như hình dưới đây

```
0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        var cars = Car.Cars;

        var list = cars.Where(f => f.YearOfManufacture >= 1990);    //  <= Truy vấn dữ liệu chưa được thực hiện

        foreach (var item in list)                                  //  <= Thực hiện truy vấn dữ liệu
        {
            Console.WriteLine(item);
        }

        /*
         *  Output:
         *
         *  1 - Honda - 1990
         *  4 - VinFast - 2017
         */
        Console.ReadKey();
    }
}
```

Sử dụng cách thực thi này sẽ giúp:

+ Cải thiện hiệu năng vì chỉ thực thi một lần.

+ Dữ liệu trả về là mới nhất.

LINQ To Entity Framework

- **Chú ý: có 2 kịch bản truy vấn**
  - Trì hoãn thực thi: where, việc thực thi truy vấn chỉ khi thực hiện foreach
  - Thực thi ngay lập tức (Immediate Execution): thực thi ngay lập tức câu lệnh và trả về kết quả. Để thực thi ngay lập tức, chúng ta có thể dùng những toán tử chuyển đổi như ToList, ToArray, ToDictionary... hoặc những toán tử thành phần như First, FirstOrDefault, Last, LastOrDefault...

LINQ Extension Methods (3)

- Cú pháp Cập nhật dữ liệu trong LinQ to Entity Framework

LINQ Extension Methods		
Function	**Syntax**	**Example**
Insert data	.Add (object)	_contextDB.Nhanviens.Add(nv1)
Edit data	.Update<table>(object)	_contextDB.Update<Nhanviens>(nv1)
Delete data	.Remove(object)	_contextDB.NhanViens.Remove(nv1)
Save database: **db.SaveChanges()**		

Hướng dẫn thực hành LINQ to Entity

- Sử dụng EF theo mô hình Database First và DB News

1. Restore DB News
2. Setup Entity Framework
3. Tạo Entity Framework sử dụng mô hình DB First
4. Tạo ADO.NET Data Model
5. Tạo mới Form thao tác với table Bài viết
    1. Lấy danh sách tất cả bài viết
    2. Lấy danh sách bài viết thuộc thể loại TL02
    3. Lấy 5 bài viết mới nhất

- Tạo mới Controller thao tác với table Thể loại
    - Insert thể loại sách mới
    - Edit thể loại sách
    - Delete thể loại sách

Bài thực hành LINQ to Entity

- Thực hiện tương tự như Demo
- Thao tác với table Tác giả
  - Hiển thị danh sách tác giả
  - Lấy 3 tác giả đầu tiên trong danh sách đã sắp xếp giảm dần theo NickName
  - Hiển thị danh sách bài viết của tác giả TG01 (sắp xếp giảm dần theo ngày đăng
  - Insert tác giả mới vào danh sách
  - Edit thông tin tác giả
  - Xóa thông tin tác giả



DISCUSSION!

tkhuong@dthu.edu.vn